

Module 6

Reinforcement Learning for Zero-Sum Differential Games

Some engineering dynamic systems and corresponding optimization problems fit into the structure of zero-sum games, which in the case of real physical systems, when games are played along trajectories of dynamic systems that change in time, the games are known as *zero-sum differential games*.

Mathematical analytical study of static games started by the end of 1920s in the works of John von Neumann. Formulation and solution of zero-sum games for dynamic systems, the so-called zero-sum differential games originated in the work of Rufus Isaacs in the 1950s, but the first collection of his research work on differential games was published a decade later in 1965 in his book titled “Differential Games” [1].

In this module, we study the zero-sum differential games in the context of reinforcement learning (RL). As is the previous modules, the presentation is done from the prospective of control systems engineering within the concept of optimal feedback decision making. Since the goal of RL is to find the optimal reward value (performance criterion value, cost value), the dynamic programming of Bellman [2] will be our main tool, similarly what was done in Module 4 for a single agent (decision maker, controller), but now with two agents (two decision makers) having antagonistic goals: what one agent gains another agent loses so that the combined reward is zero, which motivate the name to this class of games as the zero-sum games.

We will present the dynamic programming solution to the zero-sum differential games, and consider the example of the linear-quadratic (LQ) zero-sum differential game for which the analytical feedback controls (actions) can be obtained in terms of positive semidefinite solutions of a generalized algebraic Riccati equation. The policy iterations (PIs) of the RL method for the zero-sum differential games will be presented.

Zero-sum differential games appear to be an active research area in the past fifteen years, and a recent trend in the RL research community, see for example, [3]-[14] with interesting applications to diverse engineering areas. After a brief theoretical review, several RL design projects will be presented using realistic problems that appear in engineering systems.

6.1. Dynamic Programming for Zero-Sum Differential Games

Consider a general time-invariant, continuous-time, nonlinear dynamic system controlled by two agents $u_1(t)$ and $u_2(t)$ defined by

$$\frac{dx(t)}{dt} = f(x(t), u_1(t), u_2(t)), \quad x(t_0) = x_0 \quad (6.1)$$

where $f(x(t), u_1(t), u_2(t)) \in \mathbb{R}^n$ is a continuously differentiable vector function of its arguments, $x(t) \in \mathbb{R}^n$ is the system state space vector, $u_1(t) \in \mathbb{R}^{m_1}$, $u_2(t) \in \mathbb{R}^{m_2}$ are the system control input vectors (control agents), $x(t_0) = x_0$ is the system initial state, and t stands for continuous time with t_0 representing the initial time. A scalar performance criterion, to be *minimized by control agent* $u_1(t)$ and *maximized by control agent* $u_2(t)$, associated to this dynamic system, is given by

$$J(x(t_0), t_0) = \int_{t_0}^{t_f} g(x(\tau), u_1(\tau), u_2(\tau)) d\tau + \gamma(x(t_f)) \quad (6.2)$$

where $g(x(\tau), u_1(\tau), u_2(\tau))$ is a *nonnegative scalar function* continuously differentiable with respect to its arguments, representing the value of the performance criterion (cost) at the continuous time instant t (instant cost), t_f is the final time instant assumed to be fixed, and $\gamma(x(t_f))$ is the final time constraint imposed on the state variables. Since the goals of the two agents are opposite and since both of them are playing the game over trajectories of the same dynamic system, and whatever one player gains the second player loses the same amount, the game is called the zero-sum game. The optimal performance criteria for each player are

$$J_1^{opt}(x(t_0), t_0) = \min_{u_1(t)} \left\{ \int_{t_0}^{t_f} g(x(\tau), u_1(\tau), u_2(\tau)) d\tau + \gamma(x(t_f)) \right\} \quad (6.2a)$$

$$J_2^{opt}(x(t_0), t_0) = \max_{u_2(t)} \left\{ \int_{t_0}^{t_f} g(x(\tau), u_1(\tau), u_2(\tau)) d\tau + \gamma(x(t_f)) \right\} \quad (6.2b)$$

The solution to the dynamic optimization problem defined in (6.1) and (6.2) is based on the *Bellman optimality principle* of dynamic programming, which was stated as follows:

Bellman's Optimality Principle: [2], [15]. "An optimal policy has a property that whatever the initial state and initial decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision."

The goal is to find the *optimal control* inputs, preferably in the feedback form, $u_1^{opt}(x(t))$ and $u_2^{opt}(x(t))$ (optimal policies as functions of state variables) that minimize/maximize (mini-max optimization problem) the performance criterion defined in (6.2). Assuming that the solution to this mini-max optimization problem exists, the *optimal performance value* for the system that starts at state $x(t)$ with the initial time t and fixed terminal time t_f , is given by

$$\begin{aligned}
 J^{opt}(x(t), t) &= \min_{u_1(x(\tau)), t \leq \tau \leq t_f} \max_{u_2(x(\tau)), t \leq \tau \leq t_f} \left\{ \int_t^{t_f} g(x(\tau), u_1(\tau), u_2(\tau)) d\tau + \gamma(x(t_f)) \right\} \\
 &= \int_t^{t_f} g(x^{opt}(\tau), u_1^{opt}(\tau), u_2^{opt}(\tau)) d\tau + \gamma(x^{opt}(t_f))
 \end{aligned} \tag{6.3}$$

and the optimal dynamic system trajectories satisfy the system differential equation

$$\frac{dx^{opt}(t)}{dt} = f(x^{opt}(t), u_1^{opt}(x(t)), u_2^{opt}(x(t))), \quad x^{opt}(t_0) = x_0 \tag{6.4}$$

The *Bellman functional equation* for this optimization problem can be derived by using the Bellman optimality principle [16]-[17], similarly as was done for one control agent dynamic optimization problem. For the first player, it is given by

$$-\frac{\partial J^{opt}(x(t), t)}{\partial t} = \min_{u_1(x(t))} \left\{ g(x(t), u_1(t), u_2^{opt}(t)) + \left(\frac{\partial J^{opt}(x(t), t)}{\partial x} \right)^T f(x(t), u_1(t), u_2^{opt}(t)) \right\} \tag{6.5}$$

As a matter of fact, (6.5) was first derived by Isaacs in the early 1950s, and it is also known the *Isaacs-Bellman functional equation*. Similarly, the second player (decision maker, controller), can assume that the first player whose goal is to minimize the performance criterion, uses the optimal control $u_1^{opt}(x(t))$ over the entire optimization interval $[t, t_f]$, so that the second player is also faced with the classical dynamic optimization problem parametrized by $u_1^{opt}(x(t))$. Using the Bellman optimality principle, the second player comes to

$$-\frac{\partial J^{opt}(x(t), t)}{\partial t} = \max_{u_2(x(t))} \left\{ g(x(t), u_1^{opt}(t), u_2(t)) + \left(\frac{\partial J^{opt}(x(t), t)}{\partial x} \right)^T f(x(t), u_1^{opt}(t), u_2(t)) \right\} \tag{6.6}$$

Putting equations (6.5) and (6.6) together, we get the *Isaacs (Isaacs-Bellman) functional equation for zero-sum differential games*

$$-\frac{\partial J^{opt}(x(t), t)}{\partial t} = \min_{u_1(x(t))} \max_{u_2(x(t))} \left\{ g(x(t), u_1(t), u_2(t)) + \left(\frac{\partial J^{opt}(x(t), t)}{\partial x} \right)^T f(x(t), u_1(t), u_2(t)) \right\} \tag{6.7}$$

The Isaacs-Bellman functional equation can be represented in terms of a scalar function known as the Hamiltonian. The Hamiltonian of the considered dynamic optimization problem (1)-(2) is defined by

$$\begin{aligned} H\left(x(t), u_1(t), u_2(t), \frac{\partial J^{opt}(x(t), t)}{\partial x}\right) &= g(x(t), u_1(t), u_2(t)) \\ + \left(\frac{\partial J^{opt}(x(t), t)}{\partial x}\right)^T f(x(t), u_1(t), u_2(t)) &= g(x(t), u_1(t), u_2(t)) + \frac{dJ^{opt}(x(t), t)}{dt} \end{aligned} \quad (6.8)$$

Using the Hamiltonian (6.8), the *Isaacs-Bellman functional equation* becomes

$$-\frac{\partial J^{opt}(x(t), t)}{\partial t} = \min_{u_1(x(t))} \max_{u_2(x(t))} H\left(x(t), u_1(t), u_2(t), \frac{\partial J^{opt}(x(t), t)}{\partial x}\right) \quad (6.9)$$

Assume that the optimal control can be uniquely determined (preferably in the feedback form) from (6.7) by performing minimization/maximization of the right-hand side of (6.7), by using the necessary condition for minimum/maximum, and solving the corresponding algebraic equation

$$\begin{aligned} \frac{\partial}{\partial u_1} \left\{ H\left(x(t), u_1(t), u_2^{opt}(t), \frac{\partial J^{opt}(x(t), t)}{\partial x}\right) \right\} &= 0 \Rightarrow u_1^{opt} = u_1^{opt}\left(x(t), \frac{\partial J^{opt}(x(t), t)}{\partial x}\right) \\ \frac{\partial}{\partial u_2} \left\{ H\left(x(t), u_1^{opt}(t), u_2(t), \frac{\partial J^{opt}(x(t), t)}{\partial x}\right) \right\} &= 0 \Rightarrow u_2^{opt} = u_2^{opt}\left(x(t), \frac{\partial J^{opt}(x(t), t)}{\partial x}\right) \end{aligned} \quad (6.10)$$

Having obtained the expression for optimal control in (6.10), the Isaacs-Bellman functional partial differential equation that needs to be solved to find the optimal value of the performance criterion becomes

$$-\frac{\partial J^{opt}(x^{opt}(t), t)}{\partial t} = H\left(x^{opt}(t), u_1^{opt}(x(t)), u_2^{opt}(x(t)), \frac{\partial J^{opt}(x(t), t)}{\partial x}\right) \quad (6.11)$$

Equation (6.11) is known as the *Hamilton-Jacobi-Bellman-Isaacs (HJBI) partial differential equation*. The optimal trajectory $x^{opt}(t)$ can be evaluated from the system differential equation defined in (6.1) using optimal feedback controls obtained (6.10) and (6.11)

$$\frac{dx^{opt}(t)}{dt} = f(x^{opt}(t), u_1^{opt}(t), u_2^{opt}(t)), \quad x^{opt}(t_0) = x_0 \quad (6.12)$$

In optimal control theory, the gradient of the optimal performance criterion defines the so-called co-state variable $p(t) = \partial J(x(t), t) / \partial x(t)$. The differential equation for $p(t)$ can be obtained from the Hamiltonian definition formula (6.8) and by taking the partial derivative the HJBI equation (6.11) with respect to the state variable $x(t)$ leading to

$$-\frac{d}{dt}p(t) = \frac{\partial}{\partial x}\{g^{opt}\} + p^T(t) \frac{\partial}{\partial x}\{f^{opt}\} = \frac{\partial}{\partial x}H^{opt}, \quad p(t_f) = \frac{\partial J(x(t_f), t_f)}{\partial x} = \frac{\partial \gamma(x(t_f), t_f)}{\partial x} \quad (6.13)$$

In summary for the zero-sum differential game optimization problem defined by (6.1)-(6.3), the *necessary conditions for optimum* represented in terms of Hamiltonian (6.8) are given by

$$\begin{aligned} \frac{dx(t)}{dt} &= \frac{\partial}{\partial p}\{H(x(t), u_1(t), u_2(t), p(t))\}, \quad x(t_0) = x_0 \\ \frac{dp(t)}{dt} &= -\frac{\partial}{\partial x}\{H(x(t), u_1(t), u_2(t), p(t))\}, \quad p(t_f) = \frac{\partial J(x(t), t)}{\partial x}\bigg|_{t=t_f} = \frac{\partial}{\partial x}\{\gamma(x(t_f), t_f)\} \\ 0 &= \frac{\partial}{\partial u_1}\{H(x(t), u_1(t), u_2(t), p(t))\} \\ 0 &= \frac{\partial}{\partial u_2}\{H(x(t), u_1(t), u_2(t), p(t), t)\} \end{aligned} \quad (6.14)$$

It should be emphasized that in general, it is not easy to establish the *sufficient conditions* for the general zero-sum differential game optimal control problem, and that the sufficient condition for optimality can be found in the literature only for some special, for example the linear-quadratic case. In general, the sufficient condition require the Hessian matrices obtained from the Hamiltonian (6.8) be respectively positive definite (for the first agent) and negative definite for the second agent, that is

$$\begin{aligned} \frac{\partial^2}{\partial u_1^2} H(x^{opt}(t), u_1^{opt}(t), u_2^{opt}(t), p^{opt}(t)) &> 0 \\ \frac{\partial^2}{\partial u_2^2} H(x^{opt}(t), u_1^{opt}(t), u_2^{opt}(t), p^{opt}(t)) &< 0 \end{aligned} \quad (6.15)$$

Assuming that the unique optimal solutions of (6.10) exist, then plugging such solutions into the state-costate equations (6.14), a two-point boundary value problem has to be solved in order to find $x^{opt}(t)$ and $p^{opt}(t)$, that is

$$\begin{aligned}
 \frac{dx^{opt}(t)}{dt} &= f\left(x^{opt}(t), u_1^{opt}(x(t), p(t)), u_2^{opt}(x(t), p(t))\right), \quad x^{opt}(t_0) = x_0 \\
 \frac{dp^{opt}(t)}{dt} &= -\frac{\partial}{\partial x} \left\{ g\left(x^{opt}(t), u_1^{opt}(x(t), p(t)), u_2^{opt}(x(t), p(t))\right) \right\} \\
 &\quad - \frac{\partial}{\partial x} \left\{ p^{optT}(t) f\left(x^{opt}(t), u_1^{opt}(x(t), p(t)), u_2^{opt}(x(t), p(t))\right) \right\}, \quad p^{opt}(t_f) = \frac{\partial}{\partial x} \left\{ \gamma\left(x^{opt}(t_f)\right) \right\}
 \end{aligned} \tag{6.16}$$

It is not easy to solve this two-point boundary value problem in general. Moreover, since we are interested for the feedback solution, that is $u_1^{opt}(x(t))$ and $u_2^{opt}(x(t))$, the two-point value boundary value problem has to provide an analytical solution for the costate variable in terms of state variables as $p^{opt}(x(t))$, which is possible only in some special cases. However, we will see in the follow-up of this section that some approximate iterative techniques of RL, PIs or VIs can produce in the limit the approximate feedback control for some important classes of dynamic systems and performance criteria, especially for the time invariant dynamic systems.

Using the Hamiltonian (6.8) and the *Isaacs-Bellman functional equation* (6.9), in the case of time invariant systems and when the performance criterion does not explicitly depend on time (steady state optimization, infinite horizon optimization $t_f \rightarrow \infty$) equation (6.9) simplifies into

$$0 = \min_{u_1(x(t))} \max_{u_2(x(t))} H\left(x(t), u_1(t), u_2(t), \frac{\partial J^{opt}(x(t), t)}{\partial x}\right) \Rightarrow u_1^{opt}(x(t)), u_2^{opt}(x(t)) \tag{6.17}$$

and the HJBI partial differential equation becomes

$$0 = H\left(x^{opt}(t), u_1^{opt}(x(t)), u_2^{opt}(x(t)), \frac{\partial J^{opt}(x(t), t)}{\partial x}\right) \tag{6.18}$$

The state-costate equations for time-invariant systems are obtained using the four necessary conditions for optimum summarized in (6.14), which in the time-invariant case leads to

$$\begin{aligned}
 H(x(t), u_1(t), u_2(t), p(t)) &= g(x(t), u_1(t), u_2(t)) + p^T(t) f(x(t), u_1(t), u_2(t)) \\
 \frac{dx(t)}{dt} &= \frac{\partial}{\partial p} \{H(x(t), u_1(t), u_2(t), p(t))\}, \quad x(t_0) = x_0 \\
 \frac{dp(t)}{dt} &= -\frac{\partial}{\partial x} \{H(x(t), u_1(t), u_2(t), p(t))\}, \quad p(t_f) = \frac{\partial J(x(t))}{\partial x} \Big|_{t=t_f} = \frac{\partial}{\partial x} \{ \gamma(x(t_f)) \} \\
 0 &= \frac{\partial}{\partial u_1} \{H(x(t), u_1(t), u_2(t), p(t))\} \\
 0 &= \frac{\partial}{\partial u_2} \{H(x(t), u_1(t), u_2(t), p(t))\}
 \end{aligned} \tag{6.19}$$

Assuming that the unique optimal solutions of (6.17) exist, using such solutions in the state-costate equations (6.19), a two-point boundary value problem has to be solved in order to find $x^{opt}(t)$ and $p^{opt}(t)$, that is

$$\begin{aligned}
 \frac{dx^{opt}(t)}{dt} &= f(x^{opt}(t), u_1^{opt}(x(t), p(t)), u_2^{opt}(x(t), p(t))), \quad x^{opt}(t_0) = x_0 \\
 \frac{dp^{opt}(t)}{dt} &= -\frac{\partial}{\partial x} \{g(x^{opt}(t), u_1^{opt}(x(t), p(t)), u_2^{opt}(x(t), p(t)))\} \\
 &\quad - \frac{\partial}{\partial x} \{p^{optT}(t) f(x^{opt}(t), u_1^{opt}(x(t), p(t)), u_2^{opt}(x(t), p(t)))\}, \quad p^{opt}(t_f) = \frac{\partial}{\partial x} \{ \gamma(x^{opt}(t_f)) \}
 \end{aligned} \tag{6.20}$$

Since our goal is to obtain the feedback solution (very important for RL), that is $u_1^{opt}(x(t))$ and $u_2^{opt}(x(t))$, the analytical solutions for the co-state variables in terms of state variables is needed as $p^{opt}(x(t))$. This is possible in the case of a LQ zero-sum differential game.

6.2 Linear-Quadratic Zero-Sum Differential Games

Consider a linear, continuous-time, time-invariant system represented in its state space form controlled by two agents (decision makers, controllers)

$$\frac{dx(t)}{dt} = Ax(t) + B_1 u_1(t) + B_2 u_2(t), \quad x(t_0) = x_0 \tag{6.21}$$

where $x(t) \in \mathbb{R}^n$ is the system state space vector, $u_1(t) \in \mathbb{R}^{m_1}$, $u_2(t) \in \mathbb{R}^{m_2}$ are the system control input vectors (control agents), $x(t_0) = x_0$ is the system initial state, and t stands for continuous time with

t_0 representing the initial time. Matrices A, B_1, B_2 are constant and of appropriate dimensions. A scalar quadratic performance criterion, to be minimized, associated to this dynamic system, is defined by

$$J(x(t_0)) = \frac{1}{2} \int_{t_0}^{t_f} \left(x^T(t) Q x(t) + u_1^T(t) R_1 u_1(t) - u_2^T(t) R_2 u_2(t) \right) d\tau + \frac{1}{2} x^T(t_f) P_f x(t_f) \quad (6.22)$$

$$Q = Q^T \geq 0, \quad P_f = P_f^T \geq 0, \quad R_1 = R_1^T > 0, \quad R_2 = R_2^T > 0$$

where the scaling factor of 0.5 is introduced for convenience due to the fact that the derivatives of quadratic forms will be taken in the follow-up derivations. Positive semidefinite symmetric matrices Q, P_f and the positive definite symmetric matrices R_1, R_2 are weighted matrices for the state and control variables.

The Hamiltonian defined in the general form in (8), in the linear-quadratic case becomes:

$$H \left(x(t), u_1(t), u_2(t), \frac{\partial J^{opt}(x(t))}{\partial x} \right) = \frac{1}{2} \left(x^T(t) Q x(t) + u_1^T(t) R_1 u_1(t) - u_2^T(t) R_2 u_2(t) \right) + \left(\frac{\partial J^{opt}(x(t))}{\partial x} \right)^T (Ax(t) + B_1 u_1(t) + B_2 u_2(t)) \quad (6.23)$$

Using the four necessary conditions for optimum defined in (6.19), we have

$$\begin{aligned} \frac{dx(t)}{dt} &= \frac{\partial}{\partial p} \{ H(x(t), u_1(t), u_2(t), p(t)) \} = Ax(t) + B_1 u_1(t) + B_2 u_2(t), \quad x(t_0) = x_0 \\ \frac{dp(t)}{dt} &= - \frac{\partial}{\partial x} \{ H(x(t), u_1(t), u_2(t), p(t)) \} = -Qx(t) - A^T p(t), \quad p(t_f) = \frac{\partial J(x(t))}{\partial x} \Big|_{t=t_f} = \frac{\partial}{\partial x} \{ \gamma(x(t_f)) \} \\ 0 &= \frac{\partial}{\partial u_1} \{ H(x(t), u_1(t), u_2(t), p(t)) \} = R_1 u_1(t) + B_1^T p(t) \Rightarrow u_1^{opt}(p(t)) = -R_1^{-1} B_1^T p(t) \\ 0 &= \frac{\partial}{\partial u_2} \{ H(x(t), u_1(t), u_2(t), p(t)) \} = -R_2 u_2(t) + B_2^T p(t) \Rightarrow u_2^{opt}(p(t)) = R_2^{-1} B_2^T p(t) \end{aligned} \quad (6.24)$$

Note that these necessary conditions for the minimum are also sufficient conditions since $\partial^2 H / \partial u_1^2 = R_1 > 0$ and $\partial^2 H / \partial u_2^2 = -R_2 < 0$. Substituting the values of optimal controls obtained in (6.24) into the state and co-state equations, the following form of the state co-state equations resulted in

$$\begin{aligned}
 \frac{dx(t)}{dt} &= Ax(t) - B_1 R^{-1} B_1^T p(t) + B_2 R^{-1} B_2^T p(t) \\
 &= Ax(t) - (S_1 - S_2)p(t), \quad S_1 = B_1 R^{-1} B_1^T, S_2 = B_2 R^{-1} B_2^T, x(t_0) = x_0 \\
 \frac{dp(t)}{dt} &= -Qx(t) - A^T p(t), \quad p(t_f) = \frac{\partial}{\partial x} \left\{ \frac{1}{2} x^T(t_f) P_f x(t_f) \right\} = P_f x(t_f)
 \end{aligned} \tag{6.25}$$

The solution of this state-co-state two-point boundary value problem can be found by using the Riccati formalism that assumes the existence of the relationship, $p(t) = P(t)x(t)$, which leads to

$$x^T(t) \left(\frac{dP(t)}{dt} + A^T P(t) + P(t)A + Q - P(t)(S_1 - S_2)P(t) \right) x(t) = 0$$

that must be satisfied for any $x(t)$ so that the differential equation for the unknown matrix $P(t)$ becomes

$$-\frac{dP(t)}{dt} = A^T P(t) + P(t)A + Q - P(t)(S_1 - S_2)P(t), \quad P(t_f) = P_f \tag{6.26}$$

This is the *matrix Riccati differential equation for LQ zero-sum differential games*. The unique solution of the matrix Riccati differential equation exists under standard Lipchitz continuity condition. The classic book on the matrix differential Riccati equation is the book by William Reid [18]. Having obtained the solution for $P(t)$, the optimal feedback controls can be now expressed in the feedback form

$$\begin{aligned}
 u_1^{opt}(x(t)) &= -R_1^{-1} B_1^T P(t) x^{opt}(t) = -F_1^{opt}(t) x^{opt}(t) \\
 u_2^{opt}(x(t)) &= R_2^{-1} B_2^T P(t) x^{opt}(t) = F_2^{opt}(t) x^{opt}(t)
 \end{aligned} \tag{6.27}$$

where the optimal system dynamic equation is given by

$$\frac{dx^{opt}(t)}{dt} = (A - B_1 F_1^{opt}(t) + B_2 F_2^{opt}(t)) x^{opt}(t) = (A - (S_1 - S_2)P(t)) x^{opt}(t), \quad x^{opt}(t_0) = x_0 \tag{6.28}$$

with $F_1^{opt}(t)$ and $F_2^{opt}(t)$ representing the *optimal feedback gains*. It should be emphasized that the obtained optimal feedback gains in (6.27) are *time varying*, even though the original system is time invariant, which makes the optimal closed loop-system (6.28) also *time varying*.

Substitution into the Hamiltonian of the optimal control for the first player $u_1^{opt}(p^{opt}(t)) = -R_1^{-1} B_1^T p^{opt}(t) = -R_1^{-1} B_1^T \partial J^{opt}(x(t)) / \partial x(t)$ and the optimal control for the second player $u_2^{opt}(p^{opt}(t)) = -R_2^{-1} B_2^T p^{opt}(t) = -R_2^{-1} B_2^T \partial J^{opt}(x(t)) / \partial x(t)$ leads

$$\begin{aligned}
 H(x^{opt}(t), u_1^{opt}(t), u_2^{opt}(t), p^{opt}(t)) &= \frac{1}{2} \left(x^{opt}(t) Q x^{opt}(t) + u_1^{opt}(t) R_1 u_1^{opt}(t) - u_2^{opt}(t) R_2 u_2^{opt}(t) \right) \\
 &+ \left(\frac{\partial J^{opt}(x(t))}{\partial x} \right)^T \left(A x^{opt}(t) + B_1 u_1^{opt}(t) + B_2 u_2^{opt}(t) \right) \\
 &= \frac{1}{2} \left(x^{opt}(t) Q x^{opt}(t) + p^{opt}(t) (S_1 - S_2) p^{opt}(t) \right) + \left(\frac{\partial J^{opt}(x(t))}{\partial x} \right)^T \left(A x^{opt}(t) - (S_1 - S_2) p^{opt}(t) \right)
 \end{aligned}$$

with the following form of the *HBJI partial differential equation* (6.11)

$$\begin{aligned}
 -x^{opt}(t) \frac{dP(t)}{dt} x^{opt}(t) &= \frac{1}{2} x^{opt}(t) Q x^{opt}(t) + \left(\frac{\partial J^{opt}(x(t))}{\partial x} \right)^T A x^{opt}(t) \\
 -\frac{1}{2} \left(\frac{\partial J^{opt}(x(t))}{\partial x} \right)^T (S_1 - S_2) \frac{\partial J^{opt}(x(t))}{\partial x} &, \quad \frac{\partial J^{opt}(x(t_f))}{\partial x} = P_f x(t_f)
 \end{aligned} \tag{6.29}$$

It is not difficult to solve this HJBI equation. A solution of this HJBI equation can be simply guessed in a quadratic form

$$J^{opt}(x(t)) = \frac{1}{2} x^{opt}(t) P(t) x^{opt}(t), \quad \frac{\partial J^{opt}(x(t))}{\partial x(t)} = P(t) x^{opt}(t) \tag{6.30}$$

Presenting the term

$$\left(\frac{\partial J^{opt}(x(t))}{\partial x} \right)^T A x^{opt}(t) = \frac{1}{2} \left(\frac{\partial J^{opt}(x(t))}{\partial x} \right)^T A x^{opt}(t) + \frac{1}{2} x^{opt}(t) A^T \left(\frac{\partial J^{opt}(x(t))}{\partial x} \right)^T \tag{6.31}$$

and substituting (6.30) and (6.31) into the HJBI equation (6.29), it can be seen that (6.30) satisfies the HJBI equation as long as the differential Riccati equation (6.26) is satisfied. Hence, the optimal performance value for the linear-quadratic optimization problem defined by (6.21)-(6.22) is

$$J^{opt}(x(t_0)) = \frac{1}{2} x^{opt}(t_0) P(t_0) x^{opt}(t_0) = \frac{1}{2} x_0^T P(t_0) x_0 \tag{6.32}$$

To evaluate (6.29), the differential Riccati equation has to be integrated backwards in time starting at $t = t_f$ and $P(t_f) = P_f$ until the initial time $t = t_0$, the time at which $P(t_0)$ is obtained.

It is important to point out that when the linear-quadratic optimization procedure is performed over an infinite time period, $t_f \rightarrow \infty$, $P_f = 0$ then under certain controllability and observability

conditions [19], the differential Riccati equation (6.26) becomes the matrix *algebraic Riccati equation of zero-sum differential games*

$$A^T P + PA + Q - P(S_1 - S_2)P = 0 \quad (6.33)$$

This equation has the same structure as the classic algebraic Riccati equation of the linear-quadratic optimization problem. The only difference is the coefficient with respect the quadratic term, which in the case of zero-sum differential games is indefinite making equation (6.33) much harder for analysis than the classic algebraic Riccati equation where the corresponding coefficient matrix is positive semidefinite.

The constant stabilizing solution of (6.33) produces the constant value optimal feedback gains and the time-invariant optimal linear feedback system, that is

$$\begin{aligned} \frac{dx^{opt}(t)}{dt} &= \left(A - B_1 F_1^{opt} + B_2 F_2^{opt} \right) x^{opt}(t), \quad x^{opt}(t_0) = x_0 \\ F_1^{opt} &= R_1^{-1} B_1^T P, \quad F_2^{opt} = R_2^{-1} B_2^T P, \quad J^{opt} = \frac{1}{2} x_0^T P x_0 \end{aligned} \quad (6.34)$$

The algebraic Riccati equation (6.33) also appears in the frequency domain optimization in the so-called H_∞ optimization (H_∞ stands for the infinity norm in Hardy's space (the space of stable multiple-input multiple-output transfer functions; for single-input single output control systems the H_∞ norm is the peak of the magnitude Bode diagram), [20]-[21], an important area of control systems. In general, H_∞ optimization is performed in the space of stable system transfer functions with a goal to minimize the impact of the system disturbance to the system output (analog to the second player) which degrades the optimization goal of the system controller (analog to the first player), for example, to optimize a quadratic performance criterion.

6.3 RL Solution to the LQ Zero-Sum Differential Games and H_∞ Problem

In addition to the solution of the LQ zero-sum differential game, the H_∞ optimal robust control problem is also given in terms of solutions of algebraic Riccati equations with indefinite quadratic term coefficient matrices.

In general, H_∞ optimization is performed in the space of stable system transfer functions with a goal to minimize the impact of the system disturbance (represented by the second player, defined by system input $u_2(t)$) to the system output, which degrades the optimization goal of the system controller (represented by the first player, defined by system input $u_1(t)$).

Assume for the considered linear system (6.21), the system output is given by

$$y(t) = C_y x(t) + Du_1(t) \quad (6.35)$$

The H_∞ norm represents the maximum of the ratio of the signal output energy over the signal input energy (the maximal system gain), which can be analytically defined by

$$\|G_{yu_2}(s)\|_\infty = \max \left\{ \frac{\|y(t)\|_2}{\|u_2(t)\|_2}, u_2(t) \neq 0 \right\} \quad (6.36)$$

where $\|\cdot\|_2$ represents the second Euclidean norm. The essence of using H_∞ norm in control systems theory and its applications is the fact that the smaller H_∞ norm, the more robust the system is, or in other words, the system is less sensitive to input disturbances. Practically (6.36) indicates that H_∞ optimization provides the answer to the worst system disturbance case. The general H_∞ optimization problem is to find a dynamic controller whose transfer function, say $K(s)$, satisfies

$$\min_{K(s)} \|G_{yu_2}(s)\|_\infty \quad (6.37)$$

It was shown, [20]-[21], that the general H_∞ optimization problem has no unique solution, but the solution to the constrained H_∞ optimization problem defined by

$$\text{find } K(s) \text{ such that } \|G_{yu_2}(s)\|_\infty < \gamma, \quad 0 < \gamma < \infty \quad (6.38)$$

where γ is a prespecified positive number, has a unique solution. The state space solution to this frequency domain optimization problem was presented in [20]. The solution obtained has a form of an observer-driven controller [20]-[23], with the optimal observer and the optimal system feedback gains given in terms of solutions of algebraic Riccati equations with indefinite quadratic term coefficients, whose general form ($Q \geq 0, S_1 \geq 0, S_2 \geq 0$, though still positive semi-definite matrices, they have different meanings) is given by

$$A^T P + PA + Q - P \left(S_1 - \frac{1}{\gamma^2} S_2 \right) P = 0 \quad (6.39)$$

This is the same equation as the zero-sum game Riccati equation (6.33) parametrized with γ .

6.3.1 RL Algorithms for Zero-Sum Differential Game Riccati Equation

In this section, we first present algorithms commonly used for solving the algebraic Riccati equation of zero-sum differential games that is based reinforcement learning policy iterations [6]-[7], and its simplified version derived in [8]. Then, we present a convergence proof of the algorithm from [6]-[8] using reinforcement learning methodology.

Algorithm 6.1: (Lawson *et al.* [6] and Anderson *et al.* [7]).

This algorithm converges monotonically, with the quadratic rate of convergence in the neighborhood of the sought solution, to the positive semi-definite stabilizing solution of (6.33) or (6.39), assuming such a solution exists, [24]. The algorithm is formulated as follows, [6]-[7].

Define two $n \times n$ matrix sequences $P^{(i)}, Z^{(i)}, i = 0, 1, 2, \dots$, by the recursion

$$\begin{aligned} P^{(0)} &= 0 \\ A^{(i)} &= A - S_1 P^{(i)} + S_2 P^{(i)}, \quad S_1 = B_1 B_1^T, S_2 = B_2 B_2^T \\ \Phi(P^{(i)}) &= A^T P^{(i)} + P^{(i)} A + Q - P^{(i)} (S_1 - S_2) P^{(i)} \\ A^{(i)T} Z^{(i)} + Z^{(i)} A^{(i)} - Z^{(i)} S_1 Z^{(i)} + \Phi(P^{(i)}) &= 0 \\ P^{(i+1)} &= P^{(i)} + Z^{(i)}, \quad i = 0, 1, 2, \dots \end{aligned} \tag{6.40}$$

then, matrices $P^{(i)}, i = 1, 2, \dots$ form monotonically non-increasing sequence that converges to the sought solution of (6.33)

$$\begin{aligned} P^{(\infty)} &\geq \dots \geq P^{(i+1)} \geq P^{(i)} \geq \dots \geq P^{(2)} \geq P^{(1)} \geq P^{(0)} \\ A^T P^{(\infty)} + P^{(\infty)} A + Q - P^{(\infty)} (S_1 - S_2) P^{(\infty)} &= 0 \end{aligned} \tag{6.41}$$

The algorithm requires the following assumption.

Assumption 6.1: The pair (A, B_1) is stabilizable and the pair (A, C) has no unobservable modes on the $j\omega$ axis, where C is the Cholesky factor of Q , that is, $C = \text{Chol}(Q)$.

Comment 6.1: It was shown in [6]-[7] that the system feedback matrix $A^{(i)} = A - S_1 P^{(i)} + S_2 P^{(i)}$ is asymptotically stable in each iteration for $i > 1$. Algorithm 6.1, as given in (6.40)-(6.41), is computationally involved. Its simplified version was derived in [8], and presented here as Algorithm 6.2. It is obvious that Algorithm 6.2 is more in the spirit of reinforcement learning policy iterations than Algorithm 6.1. It is given in terms of standard Riccati iterations.

Algorithm 6.2: (Vrabie and Lewis [8]).

It was shown in [8], after some algebra that the need $Z^{(i)}$ for sequence can be eliminated, and only an algorithm for $P^{(i)}$ (solutions of the regular algebraic Riccati equation) can be obtained as

$$\begin{aligned} (A + S_2 P^{(i)}) P^{(i+1)} + P^{(i+1)} (A + S_2 P^{(i)}) + Q - P^{(i)} S_2 P^{(i)} - P^{(i+1)} S_1 P^{(i+1)} &= 0 \\ P^{(i)} &= 0, \quad i = 0, 1, 2, \dots \end{aligned} \tag{6.42}$$

Comment 1: In order to get in each iteration asymptotically stable system feedback matrices $A - S_1 P^{(i)} + S_2 P^{(i)}$, going through a sequence of algebraic Riccati equations (6.42) of Algorithm 6.2, that will eventually converge to an asymptotically stable matrix $A - S_1 P^{(\infty)} + S_2 P^{(\infty)}$, the free term $Q - P^{(i)} S_2 P^{(i)}$ in (6.45) must be positive semidefinite $\forall i$, and the corresponding Cholesky factors of $Q - P^{(i)} S_2 P^{(i)}$ form detectable pairs [25] with feedback matrices $A + S_2 P^{(i)}$ in each iteration. Note that it was shown in both [6] and [7] that the matrix $A - S_1 P^{(i)} + S_2 P^{(i)}$ is asymptotically stable for every $i > 1$, and that the matrix $Q - P^{(i)} S_2 P^{(i)}$ is positive semidefinite and has no unobservable modes on $j\omega$ axis (which supersedes the required detectability condition).

RL Interpretation of Algorithm 6.1

It was presented in [6]-[7] that Algorithm 6.1 can be interpreted using RL. Here, we provide the convergence proof of the simplified version of Algorithm 6.1, as given as Algorithm 6.2 of [8], via the RL approach to the linear-quadratic optimization problem defined in (6.21) and (6.22) for steady state optimization when $t_f \rightarrow \infty$. Starting with the initial approximation $u_2^{(0)}(t) = 0$, and successively (iteratively $i = 0, 1, 2, \dots$) performing optimization of the approximate version of the performance criterion (6.22), now defined by

$$J^{(i)}(x(t_0), u_1(t), u_2^{(i)}(t)) = \min_{u_1, u_2} \left\{ \frac{1}{2} \int_{t_0}^{\infty} \left(x^T(t) Q x(t) + u_1^T(t) R_1 u_1(t) - u_2^{(i)T}(t) R_2 u_2^{(i)}(t) \right) dt \right\} \quad (6.43)$$

$$Q = Q^T \geq 0, \quad P_f = P_f^T \geq 0, \quad R_1 = R_1^T > 0, \quad R_2 = R_2^T > 0$$

along the trajectories of the approximate dynamic system

$$\frac{dx^{(i)}(t)}{dt} = Ax^{(i)}(t) + B_1 u_1(t) + B_2 u_2^{(i)}(t), \quad x^{(i)}(t_0) = x_0 \quad (6.44)$$

leads to the approximate optimal solution for the first player (who minimizes the performance criterion) as $u_1^{(i+1)}(t) = -R_1^{-1} B_1^T P^{(i+1)} x^{(i)}(t)$. Then, the second player (who maximizes the performance criterion) approximate optimal strategy is updated to $u_2^{(i+1)}(t) = R_2^{-1} B_2^T P^{(i+1)} x^{(i)}(t)$ and the optimization process is iteratively continued. The reinforcement learning strategy includes optimization in each step, and it can be performed as explained in the proceeding, which in fact constitutes a reinforcement learning convergence proof of the proposed strategy to the optimal saddle point solution (assuming it exists). The approximate performance criterion and the approximate system under the second player approximate strategy $u_2^{(i)} = R_2^{-1} B_2^T P^{(i)} x^{(i)}(t)$ in the iteration number i become

$$J^{(i)}(x(t_0), u_1(t)) = \min_{u_1} \left\{ \frac{1}{2} \int_{t_0}^{\infty} \left(x^{(i)T}(t) (Q - P^{(i)} B_2 R_2^{-1} B_2^T P^{(i)}) x^{(i)}(t) + u_1^T(t) R_1 u_1(t) \right) dt \right\} \quad (6.45)$$

along the trajectories of the approximate dynamic system

$$\frac{dx^{(i)}(t)}{dt} = (A + B_2 R_2^{-1} B_2^T P^{(i)}) x^{(i)}(t) + B_1 u_1(t), \quad x^{(i)}(t_0) = x_0 \quad (6.46)$$

Forming the Hamiltonian in the iteration number i , we have

$$H^{(i)} = \frac{1}{2} \left(x^{(i)T}(t) (Q - P^{(i)} B_2 R_2^{-1} B_2^T P^{(i)}) x^{(i)}(t) + u_1^T(t) R_1 u_1(t) \right) + p^{(i)T} \left[(A + B_2 R_2^{-1} B_2^T P^{(i)}) x^{(i)}(t) + B_1 u_1(t) \right] \quad (6.47)$$

Optimizing this approximate Hamiltonian by finding $\partial H^{(i)} / \partial u_1 = 0$, we obtain $u_1^{(i+1)}(t) = -R_1^{-1} B_1^T P^{(i+1)} x^{(i)}(t)$, which after substitution into the approximate performance criterion and the approximate system differential equation produces

$$J_{opt}^{(i,i+1)}(x(t_0)) = \frac{1}{2} \int_{t_0}^{\infty} \left(x^{(i)T}(t) (Q - P^{(i)} B_2 R_2^{-1} B_2^T P^{(i)} + P^{(i+1)} B_1 R_1^{-1} B_1^T P^{(i+1)}) x^{(i)}(t) \right) dt \quad (6.48)$$

$$\frac{dx^{(i,i+1)}(t)}{dt} = (A + B_2 R_2^{-1} B_2^T P^{(i)} - B_1 R_1^{-1} B_1^T P^{(i+1)}) x^{(i,i+1)}(t), \quad x^{(i,i+1)}(t_0) = x_0 \quad (6.49)$$

The value of the integral (6.48) along trajectories of the system (6.49) can be found by solving the following algebraic Lyapunov equation [25]

$$\begin{aligned} & \left(A + B_2 R_2^{-1} B_2^T P^{(i)} - B_1 R_1^{-1} B_1^T P^{(i+1)} \right)^T V^{(i+1)} + V^{(i+1)} \left(A + B_2 R_2^{-1} B_2^T P^{(i)} - B_1 R_1^{-1} B_1^T P^{(i+1)} \right) \\ & + Q - P^{(i)} B_2 R_2^{-1} B_2^T P^{(i)} + P^{(i+1)} B_1 R_1^{-1} B_1^T P^{(i+1)} = 0 \end{aligned} \quad (6.50)$$

Note in the algorithm of [6]-[7], the weighted matrices were selected as $R_1 = I, R_2 = I$, which implied $S_1 = B_1 B_1^T, S_2 = B_2 B_2^T$.

It is known that when optimization is performed then $V^{(i+1)} = P^{(i+1)}$ so that (6.50) transforms into

$$(A + S_2 P^{(i)})^T P^{(i+1)} + P^{(i+1)} (A + S_2 P^{(i)}) - P^{(i+1)} S_1 P^{(i+1)} + (Q - P^{(i)} S_2 P^{(i)}) = 0, \quad i = 0, 1, 2, \dots \quad (6.51)$$

The sequence of Riccati iterations (6.54) is identical to the algorithm from [6]-[7] with the initial condition $P^{(0)} = 0$. This equivalence was shown in [8]. The initial guess $P^{(0)} = 0$ is consistent with the starting control input $u_2^{(0)}(t) = 0$.

Algorithms 6.3 and 6.4: (Li and Gajic [26])

The iterative successive approximations algorithms of [26] developed for solving the LQ zero-sum differential games belong to the category of reinforcement learning algorithms like Algorithms 6.1 and 6.2, even though at the time they were developed that was not their main purpose. As a matter of fact, in [26] Li and Gajic developed two algorithms: one based on the standard algebraic Riccati equation iterations (Algorithm 6.3) that is similar to Algorithm 6.2, and another one based on solutions of a sequence of algebraic Lyapunov equations (Algorithm 6.4). Both of them converge under the following assumption.

Assumption 6.2: The triple $(A, B_1, \text{Chol}(Q))$ is stabilizable-detectable.

Algorithm 6.3:

Initialize the algorithm with the positive semi-definite solution of the standard (with the positive semi-definite quadratic term coefficient matrix) algebraic Riccati equation, whose existence is guaranteed under Assumption 6.3

$$A^T P^{(0)} + P^{(0)} A + Q - P^{(0)} S_1 P^{(0)} = 0 \quad (6.52)$$

Run the following standard algebraic Riccati iterations

$$A^T P^{(i+1)} + P^{(i+1)} A - P^{(i+1)} S_1 P^{(i+1)} + Q + P^{(i)} S_2 P^{(i)} = 0, \quad i = 0, 1, 2, \dots \quad (6.53)$$

These iterations produces a monotonically non-decreasing sequence that converges to the positive-semi definite solution of (6.33), that is, it has the following property

$$P \geq \dots \geq P^{(i+1)} \geq P^{(i)} \geq \dots \geq P^{(1)} \geq P^{(0)} \quad (6.54)$$

Algorithm 6.4:

This algorithm has the same initialization step as Algorithm 6.3, and converges to the positive semi-definite stabilizing solution under Assumption 6.2. It is given in terms of algebraic Lyapunov iterations [27] as follows

$$(A - S_1 L^{(i)})^T L^{(i+1)} + L^{(i+1)} (A - S_1 L^{(i)}) + Q + L^{(i)} (S_1 + S_2) L^{(i)} = 0, \quad L^{(0)} = P^{(0)} \quad (6.55)$$

The algorithm has the property

$$P \geq \dots \geq L^{(i+1)} \geq L^{(i)} \geq \dots \geq L^{(1)} \geq P^{(0)} \quad (6.56)$$

and

$$L^{(i)} \geq P^{(i)}, \quad i = 1, 2, 3, \dots \quad (6.57)$$

with $P^{(i)}$ generated by Algorithm 6.3, indicating faster convergence of the algebraic Lyapunov iterations than the algebraic Riccati iterations to the positive definite stabilizing solution of the zero-sum game differential algebraic Riccati equation (6.33).

The convergence proof for this algorithm can be provided using reinforcement learning methodology as was done in the case of Algorithms 6.1 and 6.2.

There are other algorithms for solving the algebraic Riccati equation of zero-sum differential games, as reviewed in [28], including a very new algorithm developed in [28].

6.4 RL Policy Iterations for LQ Zero-Sum Differential Games Project

In this assignment students work on reinforcement learning policy iterations design project for of the linear-quadratic (LQ) (linear system and quadratic performance criterion) zero-sum differential games. The use of MATLAB and Simulink is required.

Project Format: The project report is supposed to be typed and contain standard technical paper parts: introduction, problem formulation, main results, conclusions, references. Students working on the project should include also obtained simulation results, tables, and figures. The program code should be placed at the end in the report as an appendix. Simulink block diagrams may be placed with the main body of the project report.

Project Requirements: It is required to design *two reinforcement learning policy iterations algorithms* for zero-sum LQ games using MATLAB/Simulink. Implement the RL policy iteration algorithms for *zero-sum games* based on Riccati iterations algorithm from [7], Algorithm 6.1, and the policy iterations algorithm of [26], Algorithm 6.3, given in terms of algebraic Lyapunov iterations. Use interactive operation of MATLAB scripts (or live scripts) and Simulink to simulate the system dynamics under each action. The Simulink block diagram needed for that purpose is presented in Figure 6.1.

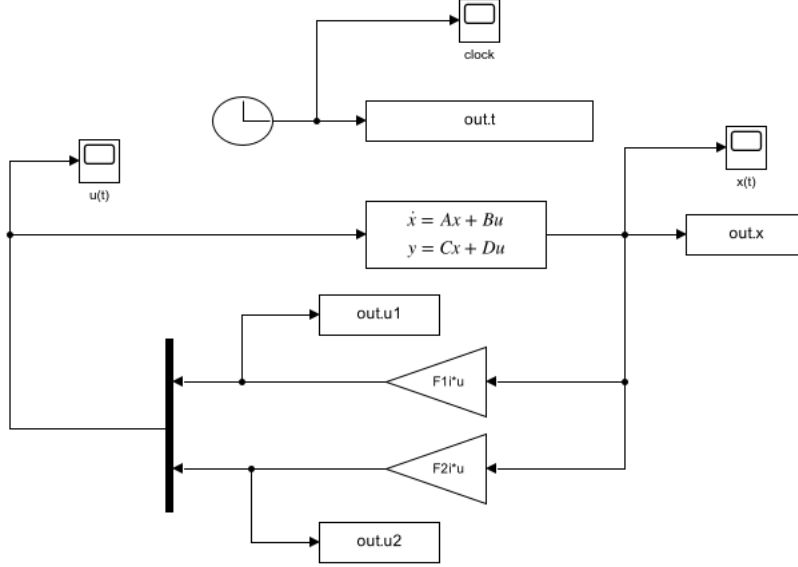


Figure 6.1: Simulink block diagram for RL zero-sum differential games

For the considered policy iteration RL algorithm use Example 1 from [6], whose state space matrices for a linear, continuous-time, time-invariant system controlled by two agents (decision makers), are given by:

$$A = \begin{bmatrix} -3.4573 & -0.0313 & 0.1167 & 0.1295 \\ 0.6203 & -1.9884 & 1.9267 & 0.2827 \\ -1.8066 & 1.9929 & -3.4093 & -0.4120 \\ -0.3954 & 0.3908 & 0.4544 & -5.1381 \end{bmatrix}, \quad C = \begin{bmatrix} -1.6758 & -0.4228 & 2.1930 & 0.8601 \\ 0.6654 & 0.9273 & -2.0392 & -1.3478 \\ -0.7585 & 0.1406 & 0.9184 & 0.7515 \\ 0.3357 & -0.0278 & 0.2078 & 0.7607 \end{bmatrix}$$

$$R_1 = I_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad R_2 = I_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad Q = C^T C$$

$$B_1^T = \begin{bmatrix} -1.6178 & -1.0728 & 0.8247 & 0.7092 \\ -1.0622 & 1.0278 & 0.6979 & 0.6806 \end{bmatrix}, \quad B_2^T = \begin{bmatrix} 1.6555 & -1.4300 & 2.8250 & -1.9743 \\ 0.7164 & 0.5922 & 0.1516 & 1.5813 \\ -1.5027 & 1.4075 & -0.4710 & -1.1708 \end{bmatrix}$$

Students may also use the example from [26] and compare the results obtained for the algorithms [6]-[8] and [26] from the convergence speed point of view.

For each algorithm present three figures and one table. Plot one state variable (the one for which the initial approximation is furthest from the optimal one) and both control inputs for

iterations $k=1,2,3,4,5$ (or other iterations in the case of slow convergence). In the table provide the approximate optimal performance criterion per iteration showing its convergence to the optimal values.

Hint 1: Study the sample MATLAB script presented in the follow-up.

6.5 Electric Car Powered by a Fuel Cell Policy Iterations Demo

Fuel cells are electrochemical devices that produce electricity through the process reverse to water electrolysis. They combine hydrogen and oxygen to produce water and electricity. Among several types of fuel cells, the most developed and the best understood are proton exchange membrane (PEM) full cells. They are used these days for vehicular and domestic applications. A 10-cm cubic fuel cell produces around 5 kW, which is sufficient to power a house. A 20-cm cubic fuel cell produces 80 kW of electric power, which is sufficient to power an electric car. These days hydrogen energy fuel cell based passenger electric cars are produced by the several leading car manufactures, for example, 2024 Hyundai Nexo, 2025 Toyota Mirai, 2026 Toyota bZ, 2025 Honda CR-Ve:FCEV.

The mathematical model of a hydrogen gas reformer (also known as a hydrogen processing system) produces hydrogen from hydrogen rich fuels (gasoline, natural gas) to be used by a PEM fuel cell, [30]-[32]. It has a pretty interesting structure in which the fuel cell produced electric current represents a disturbance to the state variables of the hydrogen processing system. The disturbance can not be completely and instantly rejected at the point of jump discontinuities since time is needed for the transient caused by disturbance changes to settle down, as was demonstrated in [32]. The corresponding control problem can be modelled as a zero-sum differential LQ game problem with the disturbance (produced electric current) playing a role of the antagonistic player.

The hydrogen gas reformer state space and penalty matrices are given by [30]-[32]

$$A = \begin{bmatrix} -0.074 & 0 & 0 & 0 & 0 & 0 & -3.53 & 1.0748 & 0 & 0 \\ 0 & -1.468 & -25.3 & 0 & 0 & 0 & 0 & 0 & 2.5582 & 13.911 \\ 0 & 0 & -156 & 0 & 0 & 0 & 0 & 0 & 0 & 33.586 \\ 0 & 0 & 0 & 124.5 & 212.63 & 0 & 112.69 & 112.69 & 0 & 0 \\ 0 & 0 & 0 & 0 & -3.3333 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -32.43 & 32.304 & 32.304 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 331.8 & -344 & -341 & 0 & 9.9042 \\ 0 & 0 & 0 & 221.97 & 0 & 0 & -253.2 & -254.9 & 0 & 32.526 \\ 0 & 0 & 2.0354 & 0 & 0 & 0 & 1.8309 & 1.214 & -0.358 & -3.304 \\ 0.0188 & 0 & 8.1642 & 0 & 0 & 0 & 5.6043 & 5.3994 & 0 & -13.61 \end{bmatrix}$$

$$B_1 = \begin{bmatrix} 0 & 0 & 0 & 0 & 0.12 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0.1834 & 0 & 0 & 0 & 0 \end{bmatrix}^T$$

$$B_2 = [0 \quad -0.328 \quad -0.024 \quad 0 \quad 0.0265 \quad 0.0504 \quad 0 \quad 0 \quad 0 \quad 0]^T$$

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0.994 & -0.088 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$Q = 3 \times C^T C, \quad R_1 = 0.02 \times I_2, \quad R_2 = 0.1 \times I_1 = 0.1$$

MATLAB RL-Design-Project Program 6.1:

```
% RL for a Zero-Sum Game for a Hydrogen Processing System
clear all; close all; clc
% Input Data: State Space Matrices
A11=[-0.074 0 0 0 0;
      0 -1.468 -25.3 0 0;
      0 0 -156 0 0;
      0 0 0 124.5 212.63;
      0 0 0 0 -3.3333];
A12=[0 -3.53 1.0748 0 0;
      0 0 0 2.5582 13.911;
      0 0 0 0 33.586;
      0 112.69 112.69 0 0;
      0 0 0 0 0];
A21=[0 0 0 0 0;
      0 0 0 0 0;
      0 0 0 221.97 0;
      0 0 2.0354 0 0;
      0.0188 0 8.1642 0 0];
A22=[-32.43 32.304 32.304 0 0;
      331.8 -334 -341 0 9.9042;
      0 -253.2 -254.9 0 32.526;
      0 1.8309 1.214 -0.358 -3.304;
      0 5.6043 5.3984 0 -13.61];
A=[A11 A12;
   A21 A22];
B1=[0 0 0 0 0.12 0 0 0 0 0;
     0 0 0 0 0 0.1834 0 0 0 0]';
B2=[0 -0.328 -0.024 0 0.0265 0.0504 0 0 0 0]';
C=[1 0 0 0 0 0 0 0 0 0;
```

REINFORCEMENT LEARNING FOR ZERO-SUM DIFFERENTIAL GAMES

```

    0 0.994 -0.088 0 0 0 0 0 0 0];
Q=3*C'*C;
D=zeros(2,3);
x0=[1;1;1;1;1;1;1;1;1;1];
% Controllability and Observability Tests
CO=ctrb(A,B1); rankCO=rank(CO);
OM=obsv(A,C); rankOM=rank(OM);
R1=0.02*eye(2); R1I=inv(R1);
R2=0.1; R2I=inv(R2);
gamma=1;
S=B1*R1I*B1';
Z=B2*R2I*B2'/gamma^2;
PP=are(A,S-Z,Q);
Jopt=trace(PP)
% Anderson's Algorithm
P0 = zeros(10,10);
iter = 5
for i=1:1:iter
Pi=are(A+Z*P0,S,Q-P0*Z*P0);
F1i=-R1I*B1'*Pi;
F2i=R2I*B2'*Pi;
Ji=trace(Pi)
P0=Pi;
ERRi=A'*Pi+Pi*A+Q-Pi*(S-Z)*Pi;
erri=norm(ERRi)
%
% Calling SIMULINK (interacting with environment)
model=sim(system_zs_game)
% Collecting SIMULINK results
time_itr(1:length(model.t),i) = model.t;
X_1(1:length(model.t),i) = model.x(:,1);
X_2(1:length(model.t),i) = model.x(:,2);
X_3(1:length(model.t),i) = model.x(:,3);
X_4(1:length(model.t),i) = model.x(:,4);
X_5(1:length(model.t),i) = model.x(:,5);
X_6(1:length(model.t),i) = model.x(:,6);
X_7(1:length(model.t),i) = model.x(:,7);
X_8(1:length(model.t),i) = model.x(:,8);
X_9(1:length(model.t),i) = model.x(:,9);
X_10(1:length(model.t),i) = model.x(:,10);

```

```

U_1(1:length(model.t),i,1:size(model.u1(:,:),2))=model.u1(:,:);
U_2(1:length(model.t),i) = model.u2(:);
end
%% Plotting approximate optimal states
markers = {'o','+','*','.', 'x','s','d','^','v','>','<','p','h'};
lineStyles = {'-','--',':', '-.'};
%
figure (1)
    for i=1:iter
hold on
Iname_1 = sprintf('Iteration %d', i);
plot(time_itr(:,i),X_1(:,i),...
'Marker',markers{mod(i,iter+1)},...
'linestyle',lineStyles{max(mod(i,2),1)}, 'DisplayName',Iname_1 )
    end
grid on
set(gca,'GridLineStyle','--')
xlabel('Time [s]');title('State variable x_1')
legend;
figure (2)
    for i=1:iter
hold on
Iname_2 = sprintf('Iteration %d', i);
plot(time_itr(:,i),X_2(:,i), 'Marker',markers{mod(i,iter+1)},...
'linestyle',lineStyles{2}, 'DisplayName',Iname_2)
    end
grid on
set(gca,'GridLineStyle','--')
legend; xlabel('Time [s]');title('State variable x_2')
figure (3)
    for i=1:iter
hold on
Iname_3 = sprintf('Iteration %d', i);
plot(time_itr(:,i),X_3(:,i), 'Marker',markers{mod(i,iter+1)},...
'linestyle',lineStyles{3}, 'DisplayName',Iname_3)
    end
grid on
set(gca,'GridLineStyle','--')
legend; xlabel('Time [s]');title('State variable x_3')

figure (4)
    for i=1:iter

```

REINFORCERMENT LEARNING FOR ZERO-SUM DIFFERENTIAL GAMES

```

hold on
Iname_4 = sprintf('Iteration %d', i);
plot(time_itr(:,i),X_4(:,i),'Marker',markers{mod(i,iter+1)},...
      'linestyle',lineStyles{2},'DisplayName',Iname_4)
    end
grid on
set(gca,'GridLineStyle','--')
legend; xlabel('Time [s]');title('State variable x_4')
figure (5)
    for i=1:iter
Iname_5 = sprintf('Iteration %d', i);
plot(time_itr(:,i),X_5(:,i),'Marker',markers{mod(i,iter+1)},...
      'linestyle',lineStyles{4},'DisplayName',Iname_5)
hold on
end
grid on
set(gca,'GridLineStyle','--')
legend; xlabel('Time [s]');title('State variable x_5')
figure (6)
    for i=1:iter
Iname_6 = sprintf('Iteration %d', i);
plot(time_itr(:,i),X_6(:,i))
% 'Marker',markers{mod(i,iter+1)}
'linestyle',lineStyles{max(mod(i,6),1)},'DisplayName',Iname_6)
hold on
end
grid on
set(gca,'GridLineStyle','--')
legend; xlabel('Time [s]');title('State variable x_6')
figure (7)
    for i=1:iter
hold on
Iname_7 = sprintf('Iteration %d', i);
plot(time_itr(:,i),X_7(:,i) )
    end
grid on
set(gca,'GridLineStyle','--')
xlabel('Time [s]');title('State variable x_7')
legend;
figure (8)

```

```

    for i=1:iter
    hold on
    Iname_8 = sprintf('Iteration %d', i);
    plot(time_itr(:,i),X_8(:,i) )
    end
    grid on
    set(gca,'GridLineStyle','--')
    xlabel('Time [s]');title('State variable x_8')
    legend;
    %
    figure (9)
    for i=1:iter
    hold on
    Iname_9 = sprintf('Iteration %d', i);
    plot(time_itr(:,i),X_9(:,i) )
    end
    grid on
    set(gca,'GridLineStyle','--')
    xlabel('Time [s]');title('State variable x_9')
    legend;
    %
    figure (10)
    for i=1:iter
    hold on
    Iname_10 = sprintf('Iteration %d', i);
    plot(time_itr(:,i),X_10(:,i) )
    end
    grid on
    set(gca,'GridLineStyle','--')
    xlabel('Time [s]');title('State variable x_{10}')
    legend;
    % Plotting approximate optimal controls (actions)
    figure (11)
    for i=1:iter
    Iname_u1 = sprintf('Iteration %d', i);
    plot(time_itr(:,i),U_1(:,i),'Marker',markers{mod(i,iter+1)},...
    'linestyle',lineStyles{max(mod(i,2),1)},'DisplayName',Iname_u1)
    hold on
    end
    grid on
    set(gca,'GridLineStyle','--')
    legend; xlabel('Time [s]');title('control u_1(t)')

```

REINFORCEMENT LEARNING FOR ZERO-SUM DIFFERENTIAL GAMES

```

axis([0 2 -50 200])
figure (12)
    for i=1:iter
Iname_u2 = sprintf('Iteration %d', i);
plot(time_itr(:,i),U_2(:,i),'Marker',markers{mod(i,iter+1)},...
'linestyle',lineStyles{max(mod(i,2),1)},'DisplayName',Iname_u2)
hold on
    end
grid on
set(gca,'GridLineStyle','--')
legend; xlabel('Time [s]');title('disturbance u_2(t)')
axis([0 2 -50 15])

```

The optimal and approximate performance criterion values are presented in Table 6.1, which indicates convergence of the policy iterations to the optimal value of the zero-sum game. The approximate cost is defined by

$$J^{(i)} = \text{trace}\{P^{(i)}\}$$

Since the system initial conditions are unknown, they are assumed to be uniformly distributed on a unit sphere. It can be seen that the optimal value is reached in only five iterations, indicating algorithm's quadratic convergence.

Iteration i	Approximate performance (cost) $J^{(i)}$
1	727.5902
2	750.8685
3	759.0299
4	760.2862
5	$760.2969 = J_{opt}$
6	760.2969

Table 6.1: Approximate performance (cost) for the reinforcement learning zero-sum differential game policy iterations algorithm, and its convergence to the optimal performance

Convergence of selected state trajectories to the optimal ones per iteration are presented in Figures 6.2-6.5.

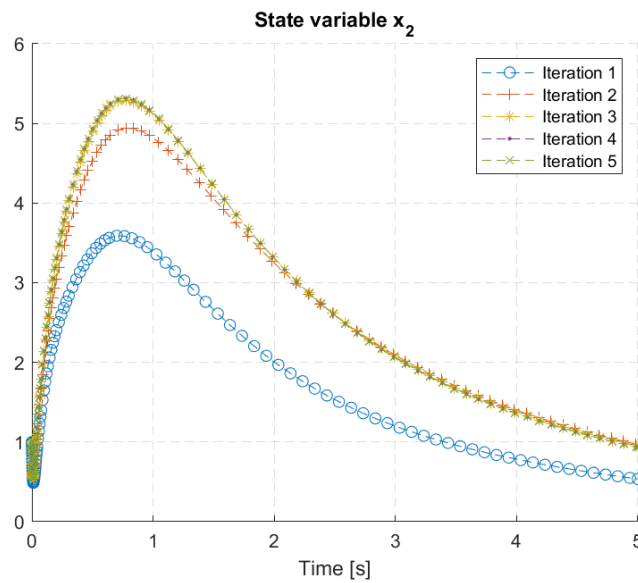


Figure 6.2: Hydrogen pressure in the anode convergence to the optimal pressure

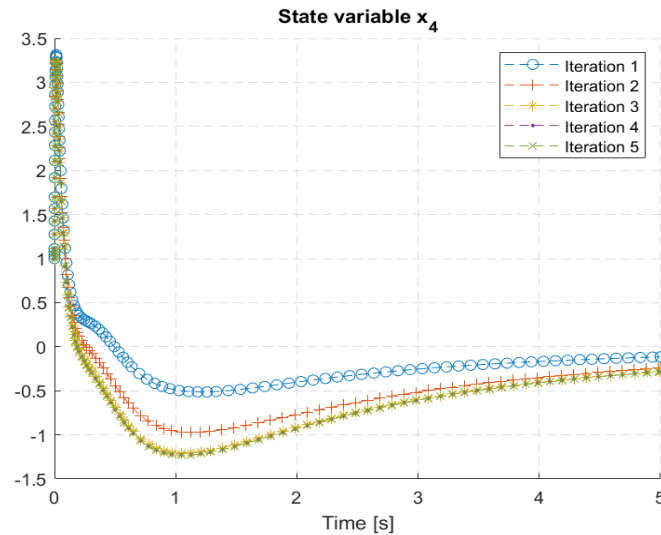


Figure 6.3: Heat exchanger pressure convergence to the optimal pressure

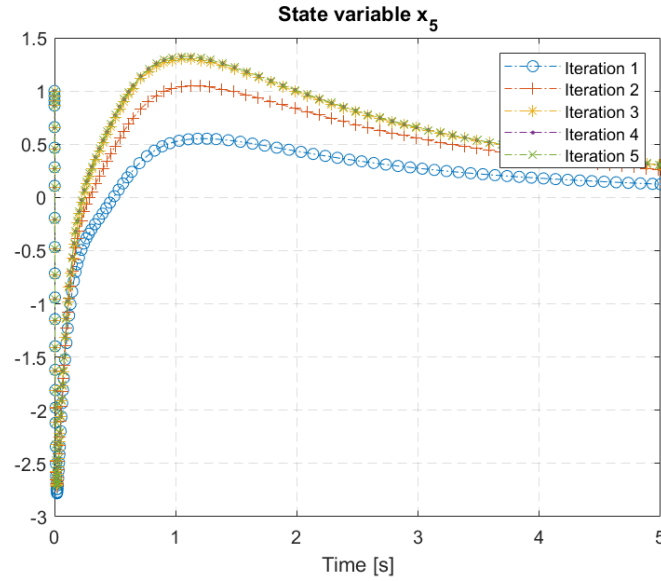


Figure 6.4: Compressor blower angular velocity convergence to the optimal one

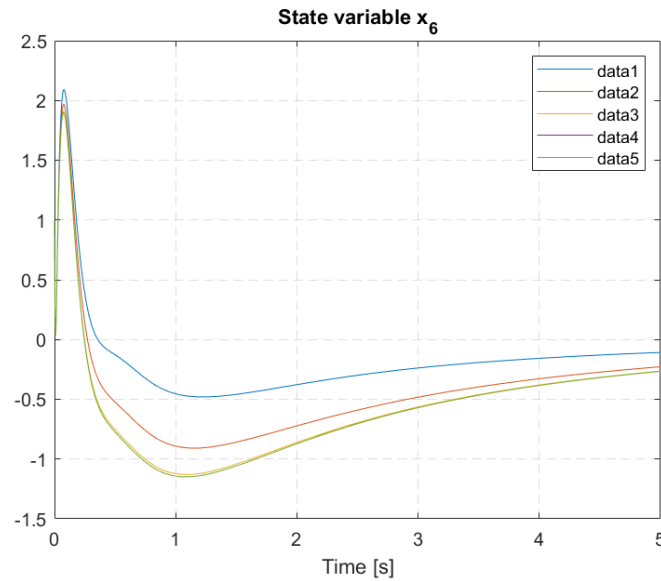


Figure 6.5: Hydro-desulfurizer pressure convergence to the optimal pressure

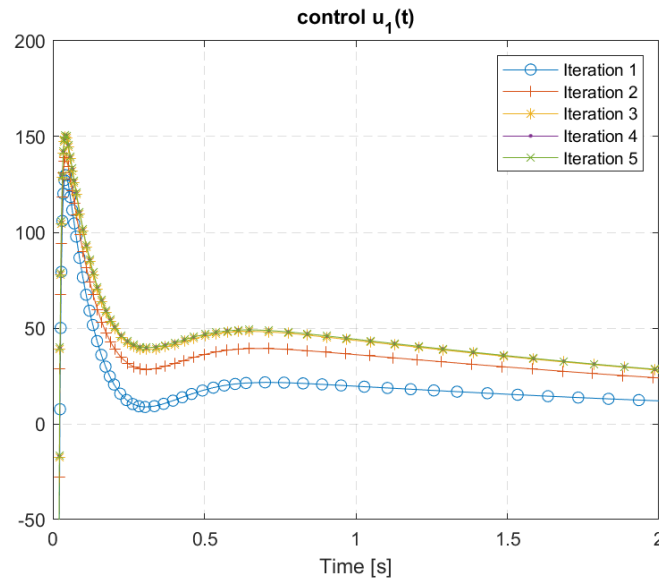


Figure 6.6: Control convergence to the optimal values

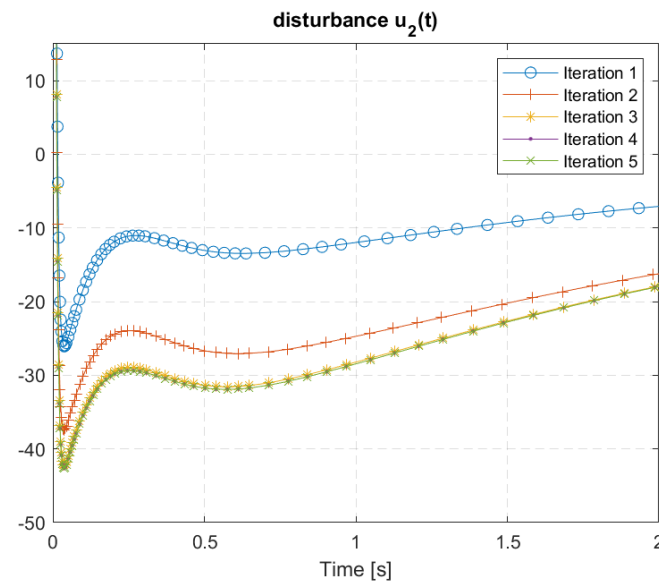


Figure 6.7: Disturbance convergence

6.6 MATLAB RL Design Projects

Project 6.1. RL Algorithm for a Power System

The following power system model was considered in [8] using the zero-sum differential game methodology in the context of H_∞ optimization (see introduction to Section 6.3 and the corresponding algebraic Riccati equation given in formula (6.42) regarding H_∞ optimization)

$$A = \begin{bmatrix} -0.0665 & 8 & 0 & 0 \\ 0 & -3.663 & 3.663 & 0 \\ -6.86 & 0 & -13.736 & -13.736 \\ 0.6 & 0 & 0 & 0 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad R_1 = R_2 = I_1 = 1, \quad \gamma = 3.5$$

$$B_1^T = [0 \quad 0 \quad 13.736 \quad 0], \quad B_2^T = [-8 \quad 0 \quad 0 \quad 0]$$

Design a policy iteration reinforcement learning optimal controller for the linear-quadratic optimization problem by following the assignment and project steps presented in Section 6.4.

Project 6.2. RL Design Algorithm for an F-16 Aircraft

Consider an F-16 aircraft defined in the state space form by [29]-[30] in the context of H_∞ optimization (see Section 6.3 regarding the relationship of zero-sum games and H_∞ optimization)

$$A = \begin{bmatrix} -1.01887 & 0.90506 & 0.00215 \\ 0.82225 & -1.07741 & -0.17555 \\ 0 & 0 & -1 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad R_1 = R_2 = I_1 = 1, \quad \gamma = 5$$

$$B_1^T = [0 \quad 0 \quad 1], \quad B_2^T = [1 \quad 0 \quad 0]$$

Design a policy iteration reinforcement learning optimal controller for the linear-quadratic optimization problem by following the assignment and project steps presented in Section 6.4.

6.7 References

- [1] Isaacs, R., *Differential Games*, Wiley, 1965.
- [2] Bellman, R., *Dynamic Programming*, Princeton University Press, Princeton NJ, 1957. Republished by Dover Publications Inc., Mineola NY, 2003.
- [3] Abu-Khalaf, M., F. Lewis, J. Huang, "Policy iterations on the Hamilton-Jacobi-Isaacs equations for H_∞ state feedback control with input saturation," *IEEE Transactions on Automatic Control*, Vol. 51, 1989-1995, 2006.
- [4] Abu-Khalaf, M., F. Lewis, and J. Huang, "Neurodynamic programming and zero-sum games for constrained control systems," *IEEE Transactions on Neural Networks*, Vol. 19, 1243-1252, 2008.
- [5] Al-Tamimi, A., F. Lewis, and M. Abu-Khalaf, "Model-free Q-learning for linear discrete-time zero-sum games with applications to H-infinity control," *Automatica*, Vol. 43, 473-481, 2007.

- [6] Lanzon, A., Y. Feng, B. Anderson, and W. Rotkowitz, "Computing the positive stabilizing solution of algebraic Riccati equations with an indefinite quadratic term via a recursive method," *IEEE Transactions on Automatic Control*, Vol. 53, 2280-2291, 2008.
- [7] Anderson, B., Y. Feng, and W. Chen, "A Game-theoretic Algorithm to Solve Riccati and Hamilton-Jacobi-Bellman-Isaac (HJBI) Equations in H_∞ Control," 277-308, in *Optimization and Optimal Control*, A. Chinchuluun *et al.* (eds), Springer, 2010.
- [8] Vrabie, D. and F. Lewis, "Adaptive dynamic programming for online solution of a zero-sum differential games," *Journal of Control Theory and Applications*, Vol. 9, 353-360, 2011.
- [9] Vrabie, D., K. Vamvoudakis, and F. Lewis, *Optimal Adaptive Control and Differential Games by Reinforcement Learning Principles*, Institution of Engineering and Technology (IET), London, United Kingdom, 2012.
- [10] Kiumarsi, B., K. Vamvoudakis, H. Modares, and F. Lewis, "Optimal and autonomous control using reinforcement learning: A survey," *IEEE Transactions on Neural Networks and Learning Systems*, Vol. 29, 2042-2062, 2018.
- [11] Lewis F. and D. Liu, Eds., *Reinforcement Learning and Approximate Dynamic Programming for Feedback Control*, IEEE Press Series on Computational Intelligence, Wiley, Hoboken, New Jersey, 2013.
- [12] Vamvoudakis, K. and F. Lewis, "On Line gaming: Real Time Solution of Nonlinear Two-Payer Zero-Sum Games using Synchronous Policy Iteration," 331-360, Chapter 18 in *Advances in Reinforcement Learning*, A. Mellouk (ed.), IntechChina, 2012.
- [13] Vamvoudakis, K., H. Modares, B. Kiumarsi, and F. Lewis, "Game theory-based control system algorithms with real-time reinforcement learning: How to solve multiplayer games on line," *IEEE Control Systems Magazine*, 33-52, 2017.
- [14] Fang, H., X. Luan, F. Liu, and Z. Ding, "Solving the zero-sum control problem for tidal turbine system: An online reinforcement learning approach," *IEEE Transactions on Cybernetics*, Vol. 53, 7635-7647, 2023.
- [15] Kirk, D., *Introduction to Optimal Control*, Prentice Hall, Englewood Cliffs, NJ, 1970.
- [16] Bellman, R., "Dynamic programming and new formalism in the calculus of variations", *Proceedings of the National Academy of Science*, Vol. 40, 231-235, 1954.
- [17] Bellman, R., "The theory of dynamic programming," *Bulletin of the American Mathematical Society*, Vol. 60, 503-516, 1954.
- [18] Reid, W., *Riccati Differential Equations*, Academic Press, 1972.
- [19] Anderson, B. and J. Moore, *Optimal Control: Linear Quadratic Methods*, Prentice Hall 1990.
- [20] Zhao, K., J. Doyle, and K. Glover, *Robust and Optimal Control*, Prentice Hall, 1996.
- [21] Doyle, J., F. Glover, P. Khargonekar, and B. Francis, "State-space solution to standard H_2 and H_∞ control problems," *IEEE Transactions on Automatic control*, Vol. 34, 831-847, 1989.
- [22] Liu K-Z. and Y. Yao, *Robust Control: Theory and Applications*, Wiley, 2016.
- [23] Wu, H-N. and B. Luo, "Simultaneous policy update algorithms for learning the solution of linear continuous-time H_∞ state feedback control," *Information Sciences*, Vol. 222, 472-485, 2013.
- [24] Chen, S., "Necessary and sufficient conditions for the existence of positive solutions to the algebraic Riccati equations with indefinite quadratic term," *Applied Mathematics and Optimization*, Vol. 26, 95-110, 1992.
- [25] Chen, T-C., *Linear Systems: Analysis and Design*, Oxford University Press, Oxford, U.K., 2012.
- [26] Li, T-Y. and Z. Gajic, "Lyapunov iterations for solving coupled algebraic Riccati equations of Nash differential games and the algebraic Riccati equation of zero-sum games," p. 333-351, in *New Trends in Dynamic Games and Applications*, G. J. Olsder (ed.), Birkhauser, 1995.

REINFORCEMENT LEARNING FOR ZERO-SUM DIFFERENTIAL GAMES

- [27] Gajic, Z., *Lyapunov Matrix Equation in System Stability and Control*, Academic Press, 1995, reprinted by Dover Publications in 2008.
- [28] Xu, L., *Learning and Control in Trustworthy and Responsible Artificial Intelligence for Cyber-Physical Systems*, Doctoral Dissertation, Rutgers University, 2024.
- [29] Vamvoudakis, K. and F. Lewis, "Online solution of nonlinear two-player zero-sum games using synchronous policy iteration," *International Journal of Robust and Nonlinear Control*, Vol. 22, 1460-1483, 2012.
- [30] Pukrushpan, J., A. Stefanopoulou, and H. Peng, *Control of Fuel Cell Power Systems: Principles, Modeling and Analysis and Feedback Design*, Springer, 2004.
- [31] Radisavljevic-Gajic, V. and P. Rose, "A new two-stage design of feedback controllers for a hydrogen gas reformer, *International Journal of Hydrogen Energy*, vol. 39, 11738-11748, 2014.
- [32] Radisavljevic-Gajic, V., M. Milanovic, and P. Rose, *Multi-Time-Scale Feedback Control of Linear Systems with Applications to Fuel Cells*, Springer 2019.